SpaceJEPA v2

A beginner's guide to the learned world model that steers the Orion capsule through reentry, and to the bigger, stronger model built next

Part 1: the pipeline and the experiments
Part 2: the models and the architecture

> **Read this first: what exists today**
>
> **v1**, the first JEPA world model (26 thousand parameters), has been trained and tested. Its results are in Part 1. **v2**, the much larger model this report is about (12.3 million parameters), is fully built and has been checked for correctness on a CPU. It has **not** been trained at full size yet. That happens when you run the notebook `ReEntryAI_JEPA_v2.ipynb` on a Colab GPU.
> So this report explains how v2 works and how it will be tested. It contains no v2 accuracy numbers. Every number in it comes from v1, from real simulator runs, or from the v2 code itself.

How figures are labeled. Each caption starts with a tag:

Tag	Meaning
REAL DATA	Flights from the simulator: the training data, or controllers flying the frozen test cases.
v1 RESULTS	Numbers measured in v1's full run (jepa/PLAN.md §9).
FROM THE CODE	Computed by the v2 code itself: attention masks, schedules, parameter counts.
ILLUSTRATION	A drawing, or a made-up example that explains an idea.
ESTIMATE	A planning number, not a measurement.

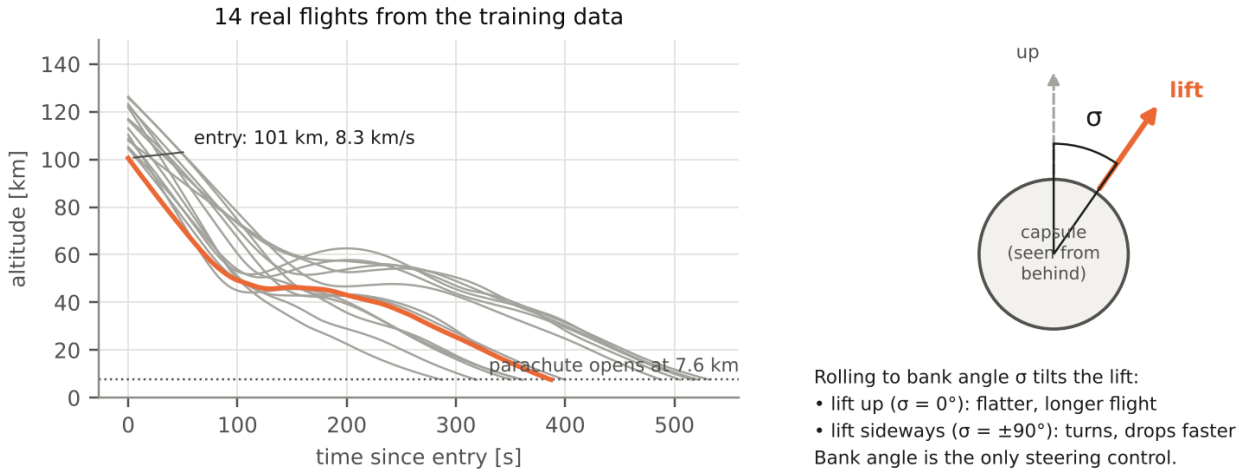
Part 1: the pipeline and the experiments

What problem we solve, how a world model is trained and used to fly, what v1 showed, and how v2 will be tested

1. The problem in one page

When the Orion capsule comes home, it hits the top of the atmosphere at about 120 km altitude and almost 8 km per second, around 25 times the speed of sound. Over the next 6 to 8 minutes, air drag slows it down. At 7.6 km altitude the small **drogue parachute** opens. The goal is to make that happen as close as possible to a chosen target point. Landing within 25 km of it counts as a success.

The capsule has no wings and no steering engine. It flies slightly tilted, so its blunt shape makes a little **lift**, about 0.3 times its drag. The only thing guidance controls is the **bank angle** σ: how far the capsule rolls around its direction of flight. Rolling points the lift in a different direction (Figure 1, right). Lift pointed up makes the flight flatter and longer. Lift pointed sideways makes the capsule turn and drop faster. Real guidance flies at some bank angle and now and then flips its sign (a **bank reversal**) to keep the capsule headed for the target.



REAL DATA **Figure 1.** Left: altitude over time for 14 flights from the v2 training data, with one highlighted. Each flight starts at the entry interface and ends when the parachute opens. Right: the bank angle σ tilts the lift. (Left: real data; right: illustration.)

Why it is hard. Nobody knows the exact air density or the capsule's exact aerodynamics on the day. If the air is 12% denser than expected, the capsule slows down sooner and falls short, unless guidance notices and corrects.

The bar to beat is the **classical predictor-corrector**. Every second it simulates the rest of the flight with a fixed, nominal physics model for several bank angles and picks the one that lands on

target. On 1,000 frozen test cases it reaches a 2.05 km median miss and 99.9% success. Its weak spot is that its model never changes: it does not learn what the air is really like on this flight.

2. The idea: learn a model of the world, then plan with it

Three earlier attempts in this project learned a **policy**, a function that maps the current state straight to a bank command. All three fell far short:

Approach	What went wrong	Result
Reinforcement learning (PPO) from scratch	never discovered the bank-reversal pattern by trial and error	~1% success
Behavior cloning (copy the classical controller)	small mistakes take it to states it never saw, where it is lost	91 km median miss
Cloning, then PPO	the RL step destroyed the cloned skill	438 km median miss

The new approach is adapted from the SkyJEPA paper (Rao et al., 2026, quadrotors). It learns a **world model** instead: a function that answers "if I fly this sequence of bank angles from here, what happens?" A **planner** then uses it like a flight simulator in its head. Every 5 seconds it tries 256 candidate plans inside the model, and flies the one predicted to land closest to the target.

> **In plain words: why a world model helps**
>
> It can be learned from **any** flights, even random ones, because it only has to predict what happens next.
> It needs no reward and no trial-and-error exploration.
> Mistakes do not snowball: the planner looks at the real state every 5 seconds and plans again.
> The target only appears in the planner's score, so the same model could fly to any target.

What "JEPA" means

JEPA stands for **Joint-Embedding Predictive Architecture**. An **encoder** translates the recent history of the flight into a short list of numbers, the **latent code** (or just "code"). It is like a summary written in the model's own language. A **predictor** then learns to predict future codes from the current code and the planned bank commands. Training compares predicted codes with the codes the encoder produces for the real future (Figure 2). Because it never has to reproduce every raw number, the code is free to keep only what matters for the dynamics.

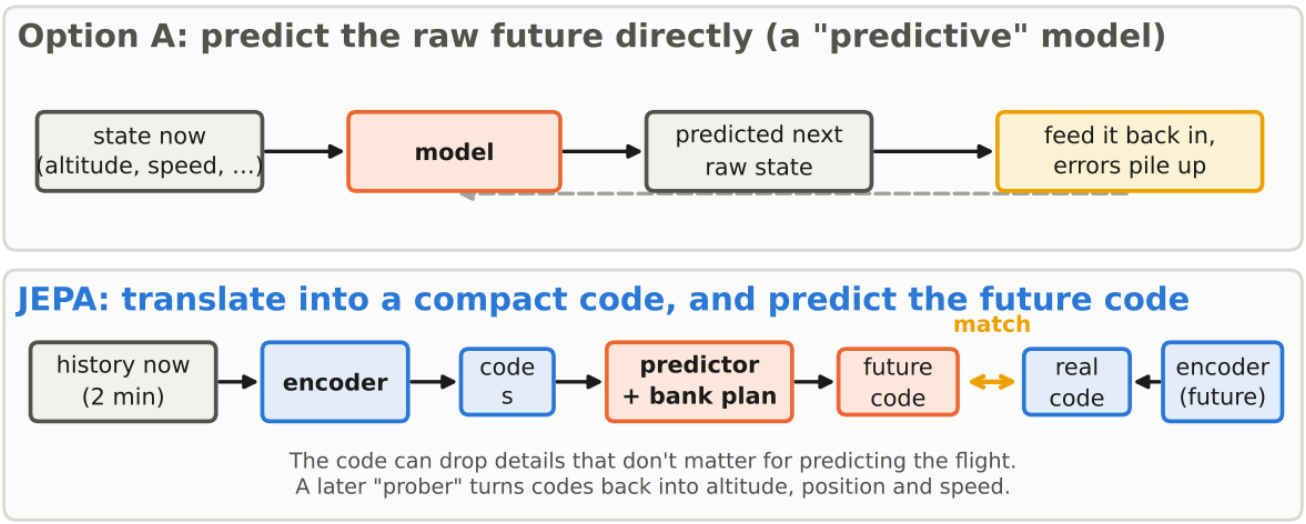


ILLUSTRATION Figure 2. Two ways to build a model that predicts the future. Top: predict the raw state and feed the prediction back in, so errors pile up. Bottom: JEPA predicts in the space of codes. A separate prober later reads physical quantities out of the codes.

v1 tested the top option as a baseline (the "predictive model"). In closed loop it reached a 13 km median miss against 1.7 km for the JEPA model (Section 7).

3. The pipeline, step by step

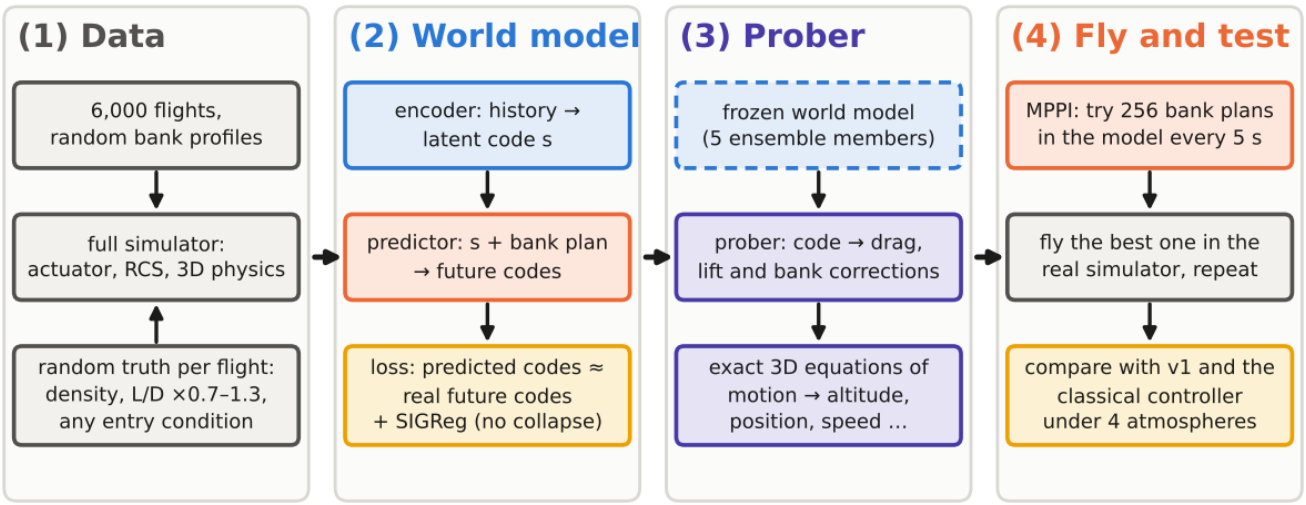


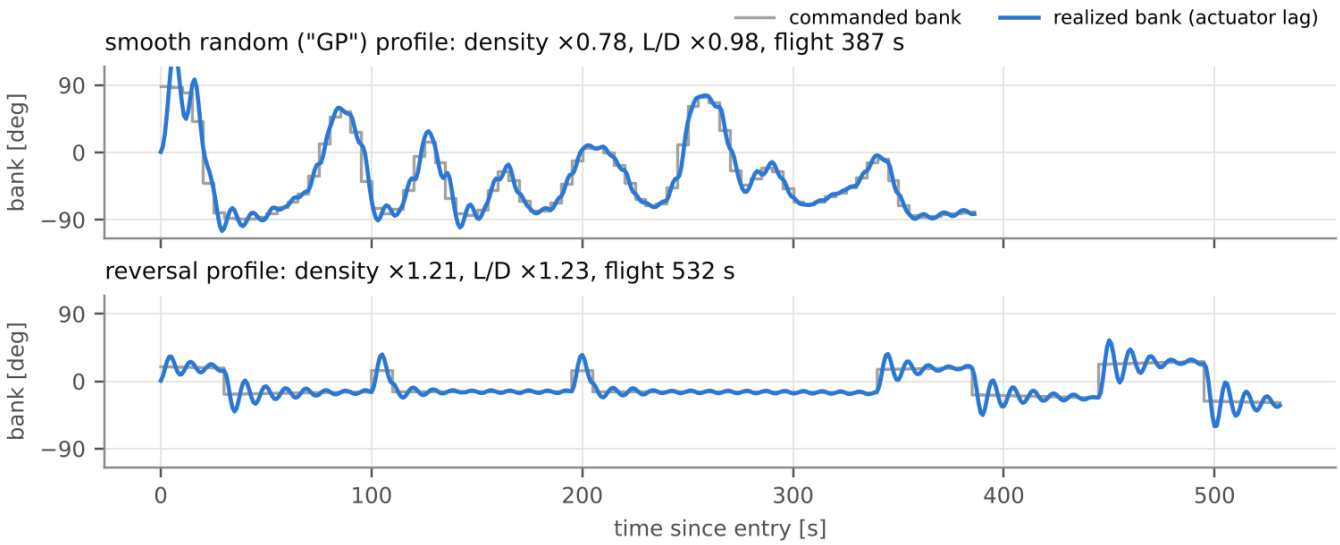
ILLUSTRATION Figure 3. The four steps of the v2 pipeline. Each step's output feeds the next. The real simulator appears twice: it makes the training data, and it flies the final tests.

1. **Data.** Fly 6,000 random flights through the full simulator, each with its own randomized atmosphere and starting point.
2. **Stage 1, the world model.** Train the encoder and predictor so that predicted codes match real future codes. No physical units are involved yet.
3. **Stage 2, the prober.** Freeze the world model, then train a small "prober" that turns predicted codes into a physical trajectory (altitude, position, speed) by correcting a simple physics model.

4. **Fly and test.** Plan with the model (MPPI), fly the frozen test cases in the real simulator, and compare with v1 and the classical controller.

4. Step 1: the training data

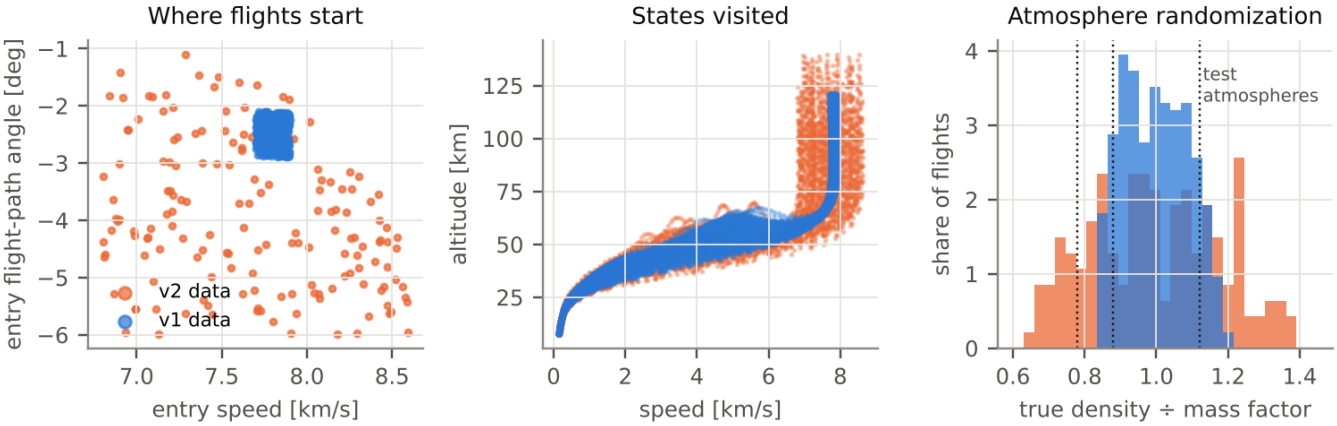
Each training flight follows a **random bank profile**, of one of two kinds. Smooth random ("GP") profiles wander over all bank angles. Reversal profiles hold a slowly changing angle and flip its sign at random times, the pattern real guidance produces. Each profile is flown through the full simulator, so the data contains everything real flight has: the bank actuator, which can only roll 20° per second; the roll controller; 12 thruster jets; 3D equations of motion; and a realistic atmosphere table. One bank command is held for 5 seconds, a **macro step**, exactly as the planner will issue them.



REAL DATA **Figure 4.** Two training flights. Gray steps: the commanded bank angle, held for 5 s each. Blue: what the capsule actually did. It lags and overshoots, because the actuator can only roll 20° per second. The world model has to learn this lag.

Every flight also gets its own **randomized truth**, so the model cannot memorize one atmosphere. It has to work out the conditions of each flight from what it observes:

- Entry point: altitude 100–140 km, speed 6.8–8.6 km/s, flight-path angle −1° to −6°, any heading.
- Air density ×0.7–1.3, lift-to-drag ratio (L/D) ×0.7–1.3, mass ×0.9–1.1. v1's data used only ×0.85–1.15 and a single entry corridor.
- The flights do not aim at the target, and most land hundreds of km from it. That is fine: the model learns physics, and the target only matters to the planner.



REAL DATA Figure 5. The v2 data (orange, 160 flights shown) against v1's data (blue, 320 flights). Left: v1's flights all started in one small box. Middle: v2 visits far more of the altitude–speed space. Right: v2's atmospheres cover all four test atmospheres (dotted) with room to spare.

In total: 6,000 flights, a median flight of about 6 minutes (75 macro steps), roughly 450,000 macro steps. The flights are split by flight: 80% for training, 10% for validation (used to choose settings), and 10% kept for testing.

5. Steps 2 and 3: training in two stages

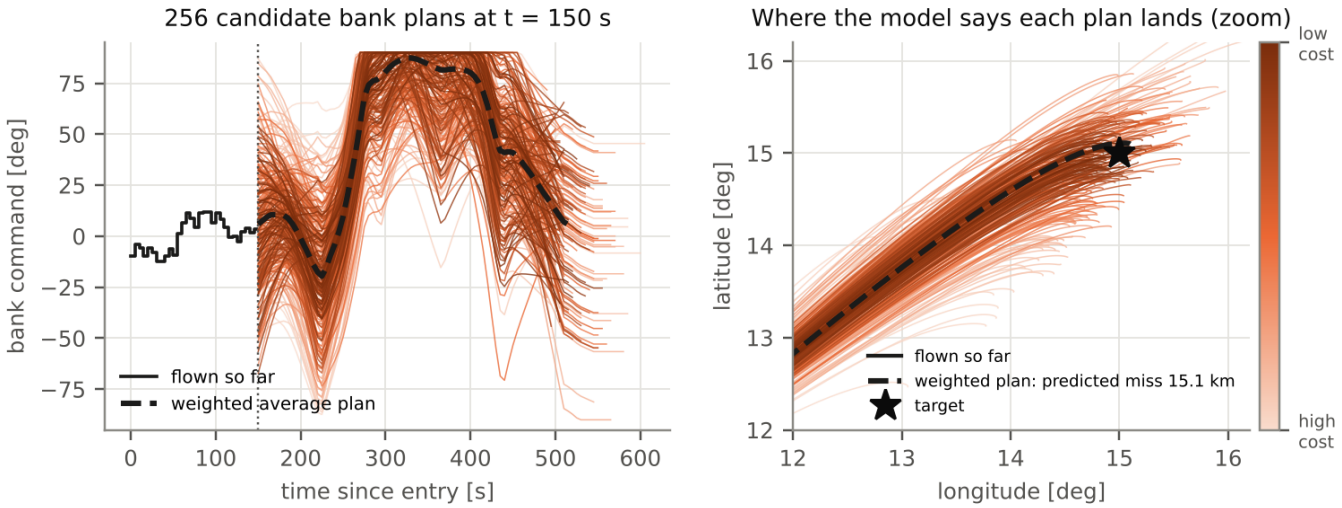
Stage 1 trains the world model alone. Its only job is to make predicted codes match the real future codes, while a regularizer called **SIGReg** stops the codes from collapsing (Part 2, Section 14). **Stage 2** freezes the world model and trains the prober on top of it.

> **In plain words: why two stages?**
> Splitting the work keeps the code general. Stage 1 learns "how flights evolve" without being told which quantities we care about. Stage 2 is then a small, cheap translation step. It also lets us test whether the code really contains information: if a prober can read the true air density out of the code, the world model has learned it.

6. Step 4: flying with the model (MPPI)

MPPI (Model Predictive Path Integral control) is a simple, sturdy way to plan with a model. Every 5 seconds:

1. Take the last 2 minutes of measured history and give it to the world model.
2. Make 256 candidate bank plans. Each is the previous best plan plus a smooth random change.
3. Fly all 256 plans inside the model down to parachute altitude, in one batch.
4. Score each plan. The score (cost, in km) is the predicted miss distance, plus small penalties for jerky commands, too many g, and diving too steeply. In v2 it also includes how much the ensemble's models disagree.
5. Average the plans, giving cheaper plans much more weight. Fly the first 5 seconds of the result, then repeat.



REAL DATA Figure 6. One real planning step of the shipped v1 model, 150 s into frozen test case 0 (normal air). Left: the 256 candidate bank plans, darker = lower cost, with the weighted average dashed. Right: where the model predicts each plan lands. At this moment the best plan is predicted to miss by 15 km, but the planner keeps replanning: the same flight, continued, opened its parachute 0.63 km from the target at t = 517 s.

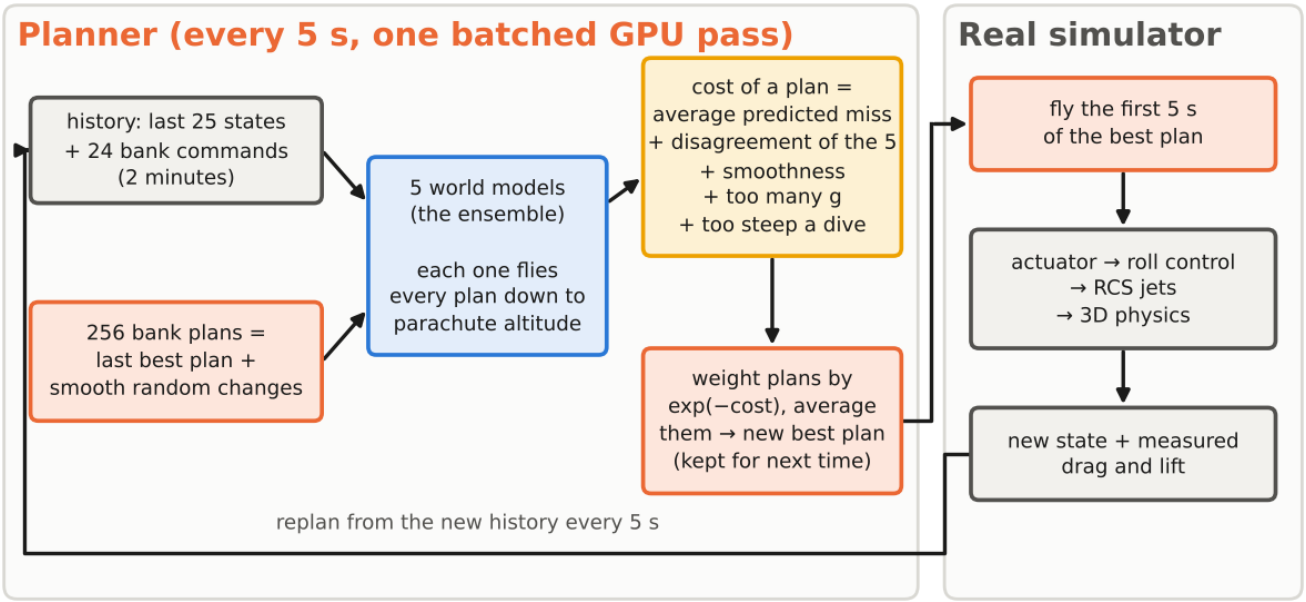


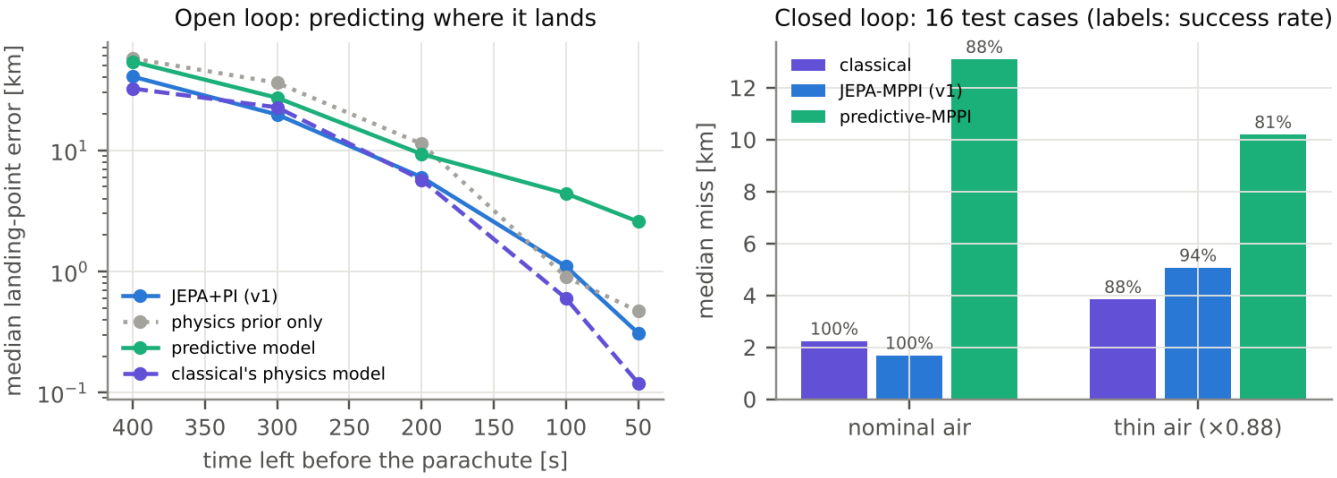
ILLUSTRATION Figure 7. The v2 planning loop. The difference from v1 is the ensemble: every plan is flown by 5 independently trained world models, and their disagreement is part of the cost.

In plain words: why there are no hand-written rules

The classical controller has rules such as "reverse the bank when the heading error is too large". MPPI has none. Bank reversals appear by themselves, because the plans that reverse at the right time are the ones predicted to land on target.

7. What v1 showed

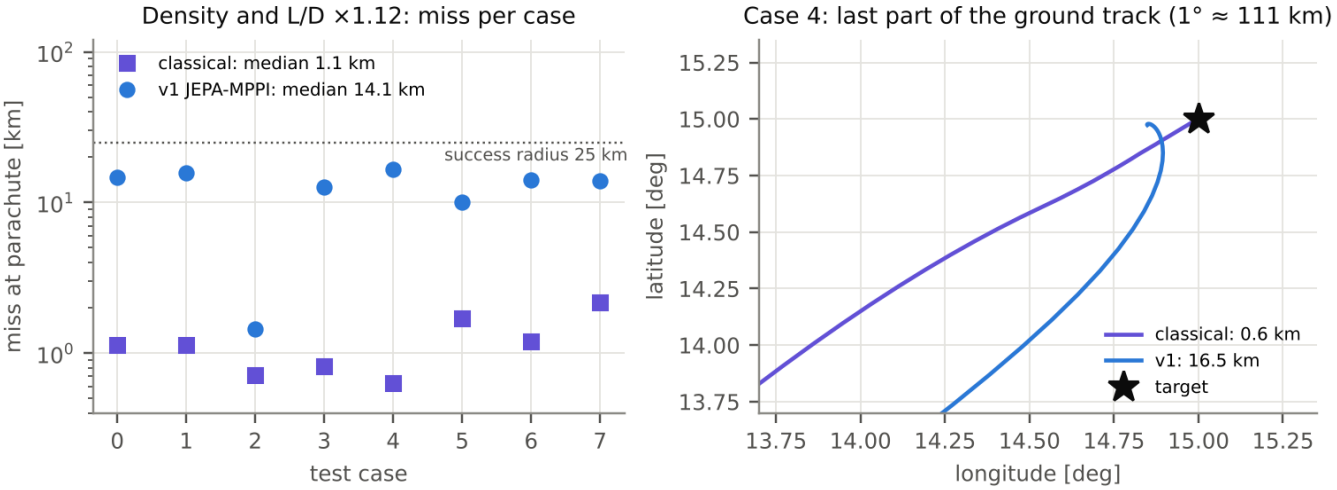
v1 (26 thousand parameters, trained on 320 narrow-corridor flights) was the first test of the whole idea. Figure 8 shows its main results.



v1 RESULTS (PLAN.md §9) Figure 8. Left: open-loop prediction of the landing point from different times before the parachute. The JEPA model plus prober beats the predictive baseline and the physics prior alone, and is close to the classical controller's own high-fidelity model. Right: closed loop on the first 16 frozen test cases; bar labels are success rates.

The good news: with 26 thousand parameters, v1 matched the classical controller in normal air (1.69 km against 2.24 km median miss, both 100% success). It was far better than every earlier learned controller, and better than planning with the predictive baseline.

The bad news: dense air. For this report, the shipped v1 model and the classical controller were flown again on the first 8 frozen test cases with density and L/D both 12% above nominal (Figure 9). The classical controller's median miss was 1.1 km (every case between 0.6 and 2.2 km); v1's was 14.1 km. Every v1 flight still landed inside the 25 km circle, but in 7 of 8 cases it landed 10–17 km off (in the remaining case, 1.4 km). Missing by about the same amount again and again is a **bias**: the model gets something about dense air systematically wrong, most likely how much drag there is.



REAL DATA Figure 9. The shipped v1 model against the classical controller, density and L/D ×1.12, 8 frozen test cases, v1's own settings (256 MPPI samples). Right: the end of the ground track for case 4, v1's worst.

Other things v1 left open:

- **No clear sign of system identification.** On flights with unusual atmospheres, v1's predictions were no better than the nominal physics model's. So there was no evidence that it inferred the air density from its history, which is the whole point of a learned model.

- **Short training horizon.** v1 learned to predict 24 steps (2 minutes) ahead, but plans 110 steps (9 minutes) ahead: 4.6 times further than it was ever trained on.
- **Narrow data.** All 320 flights started in one corridor.
- **Not g-matched.** v1 pulled about 5.6 g peak against the classical controller's 3.7 g. Part of its accuracy may have been bought with load.

8. The v2 experiment

8.1 What changes

Part 2 explains every change in detail. In one table:

	v1	v2	Aimed at
Data	320 flights, one corridor	6,000 flights, wide envelope	narrow data, dense air
Inputs	10 measured quantities	+ the drag and L/D corrections measured directly	dense-air bias
History	35 s (TCN)	120 s (attention)	system identification
Predictor	GRU (one step at a time)	transformer (all steps at once)	long horizons
Training horizon	24 steps	24 → 48 → 96 steps	extrapolation
Prober	sees the code	sees the code + the rollout's altitude and speed	dense-air bias
Models	1	ensemble of 5, disagreement used by the planner	risky plans
Parameters	32,675	12,272,655	

8.2 What the notebook runs

Notebook section	What happens	Est. time (L4 GPU)
0. Setup	clone the jepa-v2 branch, install pymsis, mount Google Drive, check the GPU	2 min
1. Data	load the 6,000 flights from Drive, split 80/10/10, plot coverage	1–2 min
2. Model	build the 5-member ensemble, print sizes and a time estimate	seconds
3. Background	start the classical-controller flights and the v1-wide retraining on the CPUs	runs in parallel
4. λ sweep	try 5 strengths of the SIGReg regularizer; keep the best one	~5 min
5. Stage 1	train the ensemble world model (24,000 steps)	~30 min
6. Stage 2	train the 5 probers (6,000 steps) and save the final model	~10 min
7. Open-loop tests	landing-point error, dense-air bias, calibration, system ID	~5 min
8. Closed loop	fly 16 test cases × 4 atmospheres × 5 controllers	~15 min
9. Verdict	score the 6 pass/fail criteria	seconds

8.3 Controls: how we will know what helped

A bigger model trained on more data could be better for many reasons. The notebook includes controls that separate them:

- **v1-wide**: v1's exact small architecture, retrained on the same 6,000 flights. If v2 beats v1-wide, the gain comes from the new model, not only from the new data.
- **v2 (no unc.)**: the same ensemble flown without the disagreement penalty. It shows what the uncertainty term buys.
- **physics-nominal**: the classical controller's own model, used only for prediction. It is the reference for open-loop accuracy.
- **Physics prior only**: the prober's crude physics with all learned corrections set to zero. It shows what the learning adds.
- **λ sweep**: the regularizer strength is chosen on validation flights, never on test flights.

8.4 The tests

Open loop (no controller; the true future commands are fed in):

- **Landing-point error vs. time-to-go**: from 400, 300, 200, 100 and 50 s before the parachute, how far is the predicted parachute point from the real one? This is the number guidance depends on.
- **Dense-air bias**: the same error split into along-track and cross-track parts and binned by true density. A bias shows up as a line that rises with density.
- **v1's home turf**: v1's own test flights, to check that the wider model has not become worse on v1's narrow corridor.
- **Calibration**: does the ensemble's disagreement grow when its error grows?
- **System identification probe**: can a straight-line (linear) readout of the code recover the true density and L/D, and how quickly after entry?

Closed loop: every controller flies the same 16 frozen test cases under four atmospheres:

Atmosphere	Density	L/D	Why
nominal	×1.00	×1.00	the classical controller's own model is exactly right
mismatch	×0.88	×0.88	thinner air
mismatch_high	×1.12	×1.12	denser air: v1's weak spot
out_of_range	×0.78	×0.80	outside v1's training range (inside v2's)

8.5 Pass/fail criteria, fixed before the run

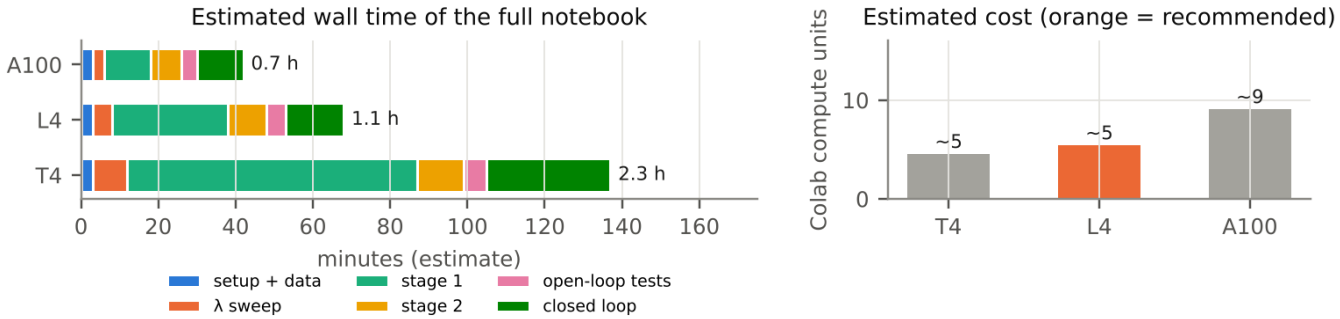
Writing down what counts as success before seeing the results stops us from moving the goalposts afterwards. The notebook's last cell scores these automatically:

#	Question	Passes if
1	Did the codes stay healthy (no collapse)?	every member's effective rank > 16 of 64
2	Is it the model, not just the data?	v2's median landing error < v1-wide's at 200 s and 300 s
3	Does it beat the classical controller's model?	v2 < physics-nominal at 200 s and 300 s
4	Is the dense-air bias gone?	mean along-track error at 200 s < 2 km, and smaller than v1's
5	Does it fly robustly?	≥ 95% success in all 4 atmospheres, and median miss in dense air ≤ classical's

#	Question	Passes if
6	Is the uncertainty useful?	spread–error rank correlation > 0.4, and the p90 miss is no worse than without the penalty in ≥ 3 of 4 atmospheres

Sixteen cases is still a small sample. Differences under about 1 km in the median should be treated as noise. Before claiming a win over the classical controller, rerun with 48 cases.

8.6 Time, cost, and which GPU



ESTIMATE Figure 10. Estimated wall time and cost of the full notebook. The estimates come from the exact training compute (33 PFLOP for stage 1) at an assumed ~15% of each GPU's peak speed, plus measured CPU costs for the simulator. Colab compute-unit rates are approximate and change; the runtime picker shows the live rate.

Recommendation: the L4 GPU. It costs about the same as a T4 but finishes in half the time. It supports the faster bf16 number format and has enough memory to skip the memory-saving mode. Its larger runtime also has more CPUs for the simulator work. Use a T4 only on the free tier. Everything is resumable, so a disconnect costs at most 10 minutes of training. The notebook prints the measured speed and the time remaining after its first few hundred training steps.

8.7 Where the results are stored

Location on Google Drive	What it holds
MyDrive/ReEntryAI_jepa_data/wide_6000/	the input: 6,000 flights (written by the data notebook)
MyDrive/ReEntryAI_jepa_v2/full/v2_ckpt.pt	the final v2 model (all 5 members and their probers)
.../full/v1_wide_ckpt.pt	the v1-wide control model
.../full/wm_stage.pt, pi_stage.pt, sweep/	training checkpoints, used to resume after a disconnect
.../full/*.csv	results: λ sweep, landing-point errors, closed-loop flights and summary, the verdict
MyDrive/ReEntryAI_jepa_v2/closed_loop_cache_full.pkl	every closed-loop flight, saved one by one

The figures are shown only inside the notebook. Use File → Save a copy in Drive after the run to keep them.

8.8 What has already been checked

This machine has no GPU, so v2 was checked on a CPU with a tiny version of the model and the first 160 real flights of the dataset. The numbers from that run say nothing about v2's real accuracy, but the checks show the code is right:

- The encoder gives the same code whether it reads a long sequence or a single 2-minute window (difference 10^{-11}).

- Running the predictor one step at a time (as v1's code expects) gives exactly the same results as running it in one parallel pass.
- v2's regularizer gives the same value as v1's on the same data.
- Both training stages lower their validation loss; the prober beats the physics prior alone.
- Checkpoints save, load, and resume correctly.
- The new GPU flight runner makes the same bank decisions as v1's unchanged runner on the same model (within 0.001°).
- The whole notebook runs from start to finish in its small "smoke" mode (under 3 minutes on 4 CPUs).

Part 2: the models and the architecture

Every block of v2: what it does, why it was chosen, and how it differs from v1

9. The big picture

v2 is an **ensemble** of 5 identical-looking but independently trained models. Each one has three parts: a state encoder, an action encoder, and a predictor (Figure 11). A fourth part, the prober, is added in stage 2. Each member has 2,454,531 parameters; the ensemble has 12,272,655.

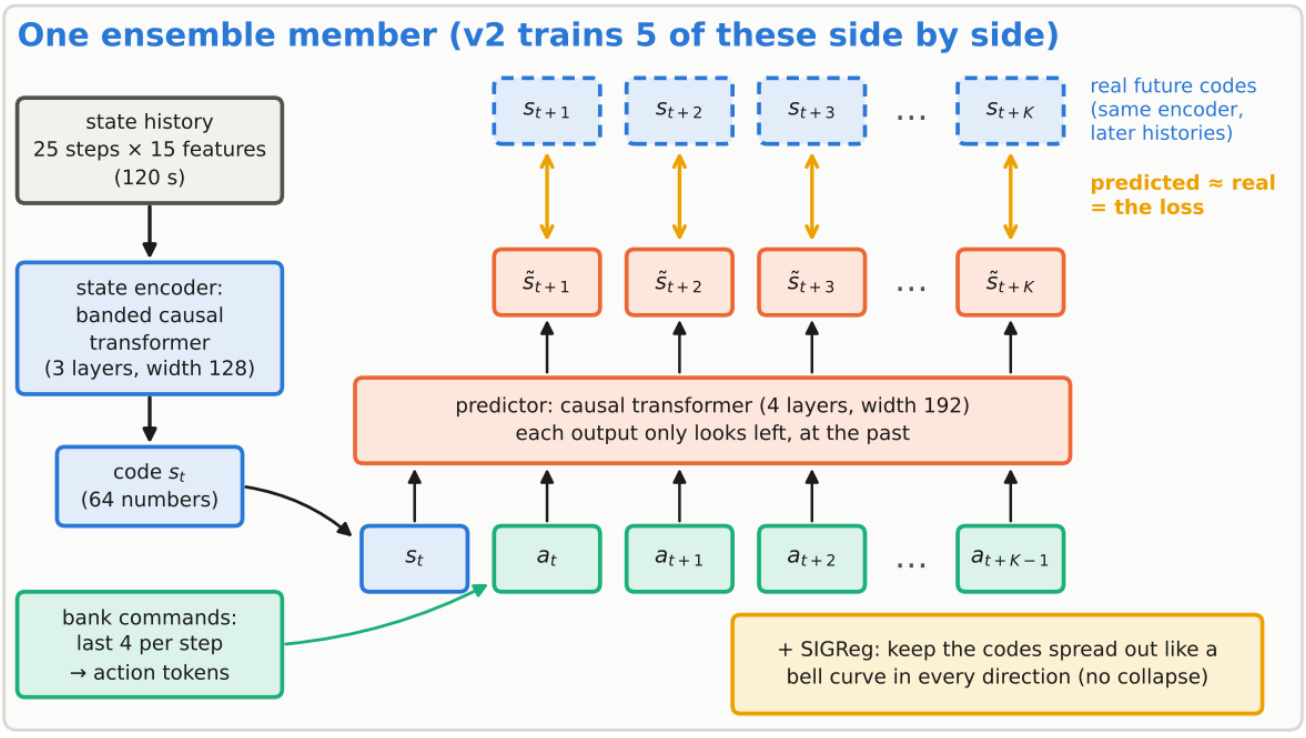


ILLUSTRATION **Figure 11.** One v2 ensemble member during stage 1. The encoder turns 2 minutes of history into a 64-number code. The predictor reads that code plus one token per planned bank command, and outputs a predicted code for every future step at once. The loss compares them with the real future codes. SIGReg keeps the codes spread out.

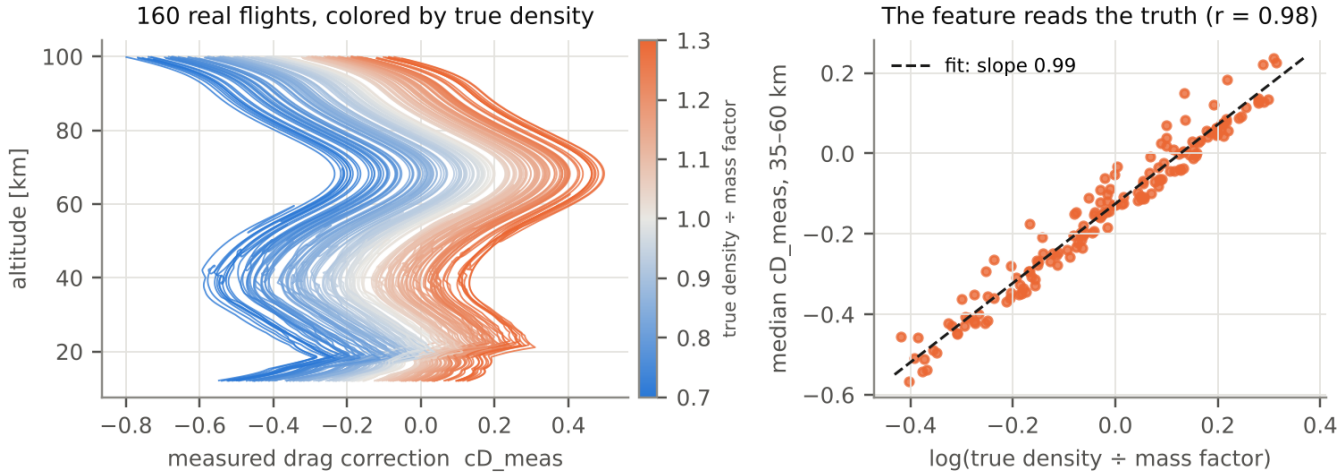
10. What the model sees: inputs

Every 5 seconds the model receives 15 numbers about the capsule. Ten are the same as v1's; five are new:

Input	Meaning	New in v2?
altitude, speed, flight-path angle γ	where the capsule is and how it moves	
sin and cos of heading, latitude	direction of flight and position on Earth	
realized bank angle, roll rate	what the capsule is actually doing (lags the command)	
drag and lift acceleration (in g)	what the accelerometer measures	
sin and cos of the bank angle	the bank's effect on lift is through sin and cos	yes
log of drag acceleration	drag grows more than 10,000× between 120 km and 30 km; the log tames that range	yes
measured drag correction $c_{D,meas}$	how much more or less drag than the simple physics prior predicts, right now	**yes**
measured L/D correction $c_{L,meas}$	how much more or less lift per drag than the prior's 0.30	**yes**

Physics-residual features: reading the air density off the accelerometer

Drag depends on air density, speed squared, and the capsule's shape and mass. The prober (Section 16) starts from a deliberately simple physics formula and learns corrections to it. v2 computes those corrections directly from the measurements: $c_{D,meas}$ = log(measured drag ÷ drag the simple formula predicts at this altitude and speed). If the air is 20% denser than the formula assumes, this number moves by about log 1.2 ≈ 0.18.



REAL DATA Figure 12. Left: the measured drag correction along 160 real training flights, colored by their true density (÷ mass) factor. The color bands separate cleanly: the feature shifts with density. The bends with altitude come from where the real atmosphere differs from the simple formula. Right: the median of the feature between 35 and 60 km against the true factor. The slope is 0.99 and the correlation r = 0.98, so the feature reads the truth almost exactly.

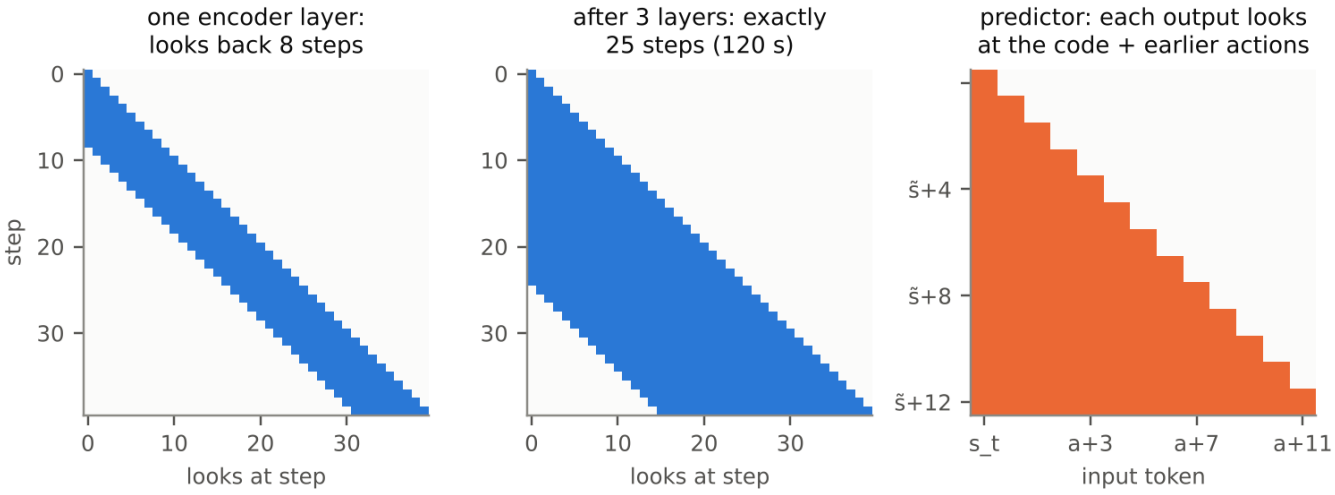
In plain words: why this targets the dense-air bias

v1 had to work out "how much denser is the air than expected" by combining altitude, speed, and drag inside a small network, a multiply-and-divide over a 10,000× range. A small, systematic slip in that calculation is exactly what a dense-air bias looks like. v2 hands the network the answer to that calculation. The network still has to decide how to use it, but it no longer has to rediscover the physics.

11. The state encoder: attention over 2 minutes

v1's encoder was a small **TCN** (temporal convolution network) that looked at the last 7 steps (35 s). v2 uses a **transformer**, the building block behind modern language models. Its key operation is **attention**: each time step looks at other time steps and decides how much each one matters. That is useful for system identification. To judge how dense the air is, the model can compare drag a minute ago with drag now, at different altitudes and speeds.

v2's attention is **banded and causal**. Each step may only look at itself and the 8 steps before it (causal: never the future). Stacking 3 such layers gives a window of exactly 25 steps = 120 s (Figure 13). Why insist on an exact window? The same encoder makes the predictor's starting code and its training targets. With an exact window, the code for "now" is the same whether the encoder reads a long training sequence or only the last 2 minutes in flight, so training and flying match.



FROM THE CODE **Figure 13.** Attention patterns computed by the v2 code (colored = allowed to look). Left: one encoder layer looks back 8 steps. Middle: after 3 layers, each step depends on exactly the last 25 steps. Right: the predictor's causal pattern: the output for future step k sees the starting code and the actions up to step k, never later ones.

Positions are encoded with **rotary embeddings** (RoPE). They tell attention how far apart two steps are, not their absolute position, so the encoder treats every 2-minute window the same way. The encoder has 3 layers, each 128 numbers wide with 4 attention heads, and outputs a code of 64 numbers (v1: 32).

12. Action tokens: packing the recent commands

The predictor needs to know the planned bank commands. v2 turns each 5-second step into one **action token**, built from that step's command and the 3 before it (20 seconds). Each command is described by its value and its sine and cosine. Why 20 seconds? Reversing the bank from +90° to −90° at the actuator's 20°/s limit takes 9 seconds, so 20 s of commands contains everything the actuator lag depends on.

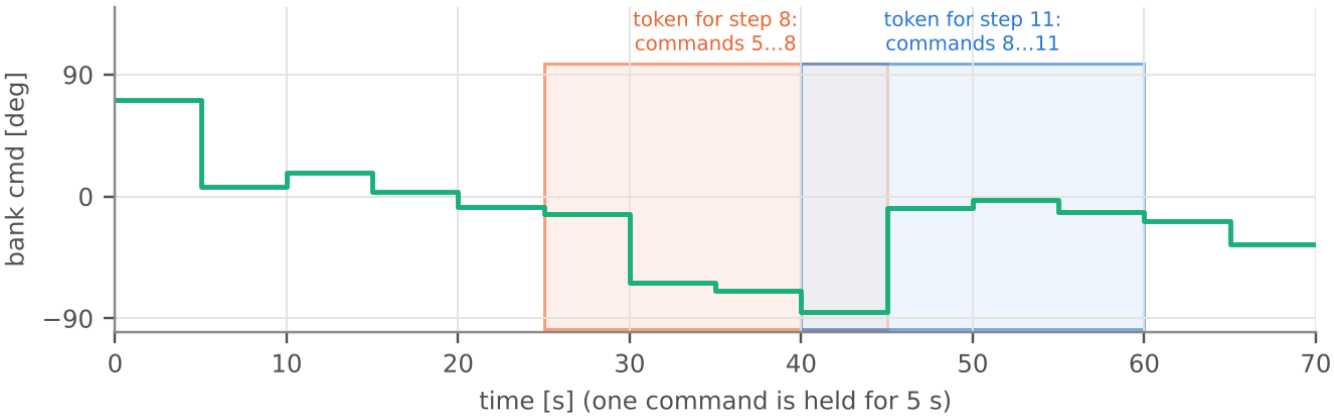


ILLUSTRATION Figure 14. Each action token packs the current command and the 3 before it. Tokens overlap, the way a sliding window does.

13. The predictor: a causal transformer

v1's predictor was a **GRU**, a recurrent network that steps forward one 5-second step at a time, so training on 96 steps means 96 steps in a row. v2's predictor is a **causal transformer**: it reads the starting code and all action tokens together and outputs every future code in one pass (Figure 15). "Causal" means each output may only look at earlier tokens. So the prediction for step k depends only on the starting code and the commands up to step k, the same rule the GRU followed by construction.

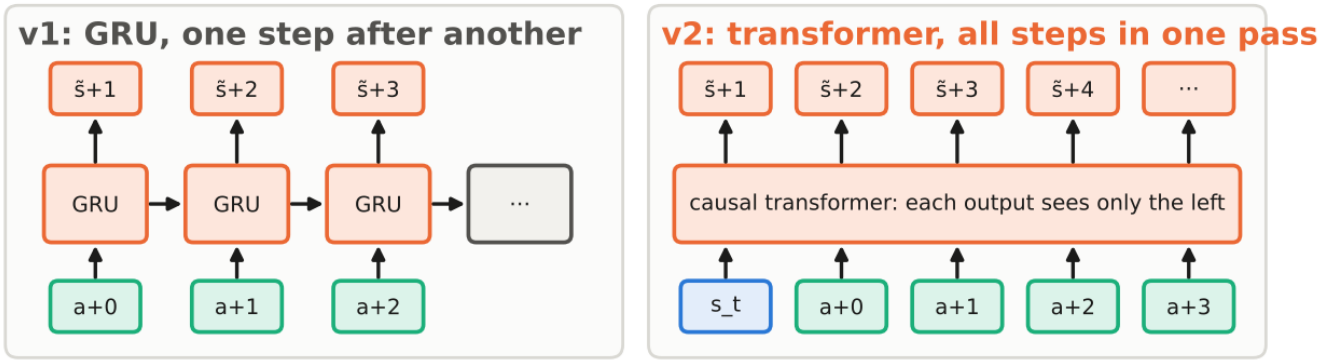


ILLUSTRATION Figure 15. v1 predicts one step after another. v2 predicts all steps in one pass. That makes long training horizons affordable on a GPU, and lets the planner score all 256 plans × 110 steps × 5 models at once.

- Size: 4 layers, 192 numbers wide, 6 attention heads, about 1.8 million parameters.
- The output is the starting code plus a learned change, and the change starts at zero. So before training, the model's guess is "nothing changes", a calm starting point.
- For compatibility with v1's evaluation and planning code, the predictor can also run one step at a time, remembering earlier steps (a "KV cache"). Both modes give identical results.

14. The training objective: prediction + SIGReg

The main loss is simple: the average squared difference between each predicted code and the real future code. But that loss alone has a cheating solution. If the encoder outputs the same code for every state, prediction is trivially perfect and the model has learned nothing. This is called **collapse**.

SIGReg (Sketched Isotropic Gaussian Regularization, from LeJEPA) prevents it. It pushes the codes across many flights to spread out like a round bell curve (a standard normal distribution) in every direction. To check this cheaply, it projects the codes onto 32 random directions and measures how far each 1-D projection is from a bell curve. That uses the Epps-Pulley test, which compares the data's "characteristic function" with the bell curve's.

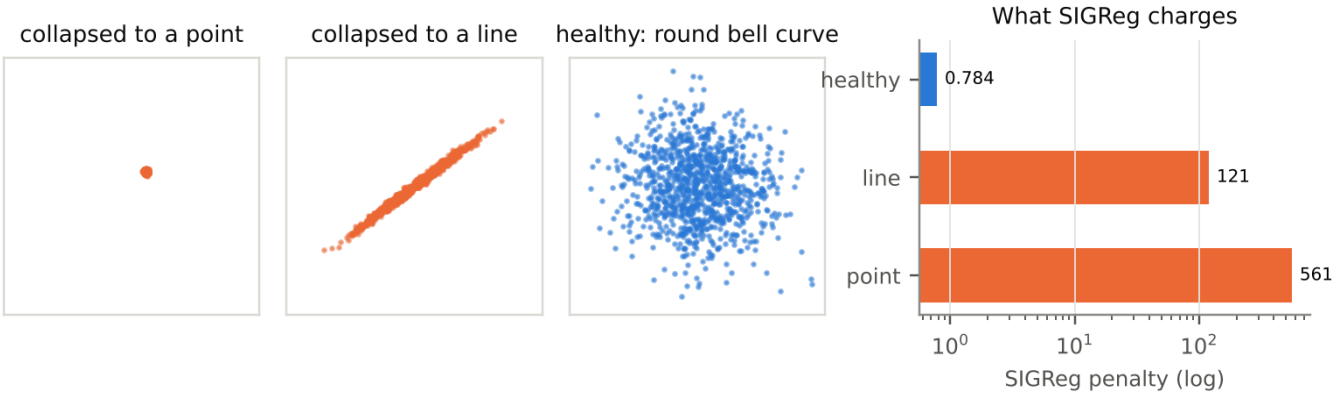
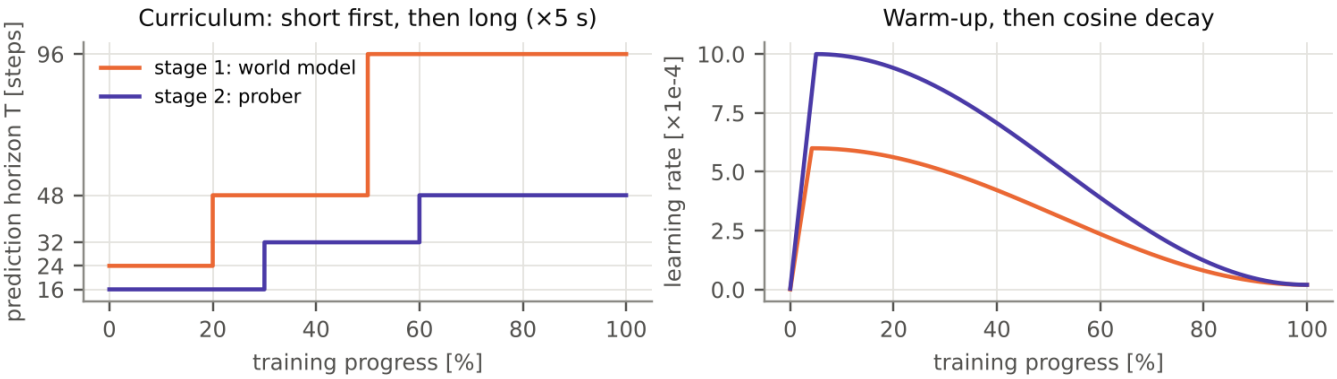


ILLUSTRATION Figure 16. Made-up 2-D codes scored by the real SIGReg code. Collapsed codes are charged heavily (point: 561, line: 121). Healthy, bell-shaped codes cost almost nothing (0.78). The scale is logarithmic.

- **Per step:** v2 applies SIGReg separately at each prediction step, as the SkyJEPA paper does. v1 pooled all steps together.
- **How strong?** The total loss is prediction + λ × SIGReg. Too small a λ allows partial collapse; too large a λ forces spread at the expense of prediction. The notebook trains 5 members with λ = 0.01, 0.02, 0.04, 0.08 and 0.16 side by side. It keeps the one whose prober predicts landing points best on validation flights.
- Like v1 and LeJEPA, v2 needs no other tricks against collapse: no stop-gradient, no slowly-updated target network, no decoder.

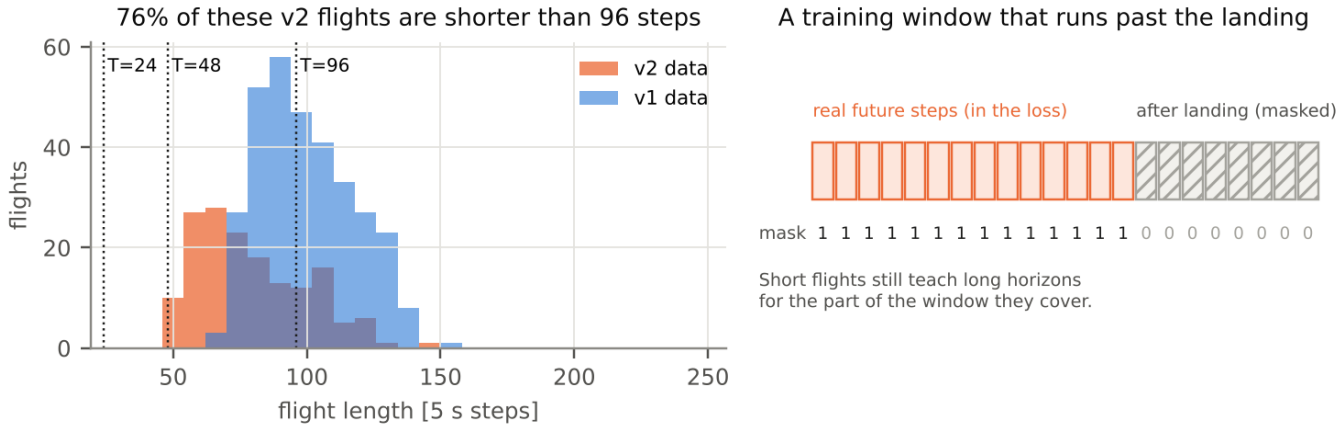
15. Long horizons: curriculum and masked windows

The planner looks up to 110 steps (9 minutes) ahead. v1 trained on 24-step windows, so it had to extrapolate 4.6× beyond its training. v2 trains on longer and longer windows: 24 steps first, then 48, then 96 (a **curriculum**, Figure 17). Starting short makes early training easy and fast; ending long matches how the model is used. The planner then extrapolates only 1.15× beyond training.



FROM THE CODE Figure 17. The training schedules used in the full notebook. Left: prediction horizon during training, for the world model (stage 1) and the prober (stage 2). Right: learning rate: a short warm-up, then a smooth cosine decay.

A long window raises a practical problem: most wide-envelope flights are shorter than 96 steps (Figure 18). Normally such flights would simply be left out of long windows, and the long-horizon training would then only see long, gliding flights, a biased sample. v2 instead lets windows run past the landing and switches the missing steps off with a **mask**.



REAL DATA Figure 18. Left: flight lengths in the v2 data (first 160 real flights) and v1's data, with the three training horizons. Right: a window that runs past the landing. Only the real steps count in the loss. (Left: real data; right: illustration.)

16. The physics-inspired prober

The prober turns predicted codes into a physical trajectory. It does not predict altitude or position directly. Instead it corrects a deliberately **crude physics prior**: one exponential atmosphere, one ballistic coefficient, L/D = 0.30, and lift pointed along the commanded bank. At each step a small network outputs three corrections: a drag factor, an L/D factor, and a bank offset (for the actuator lag). The corrected forces then go through the **exact 3D equations of motion** (Figure 19). This was v1's single biggest win: geometry and gravity come for free, and the network only learns what is hard to model.

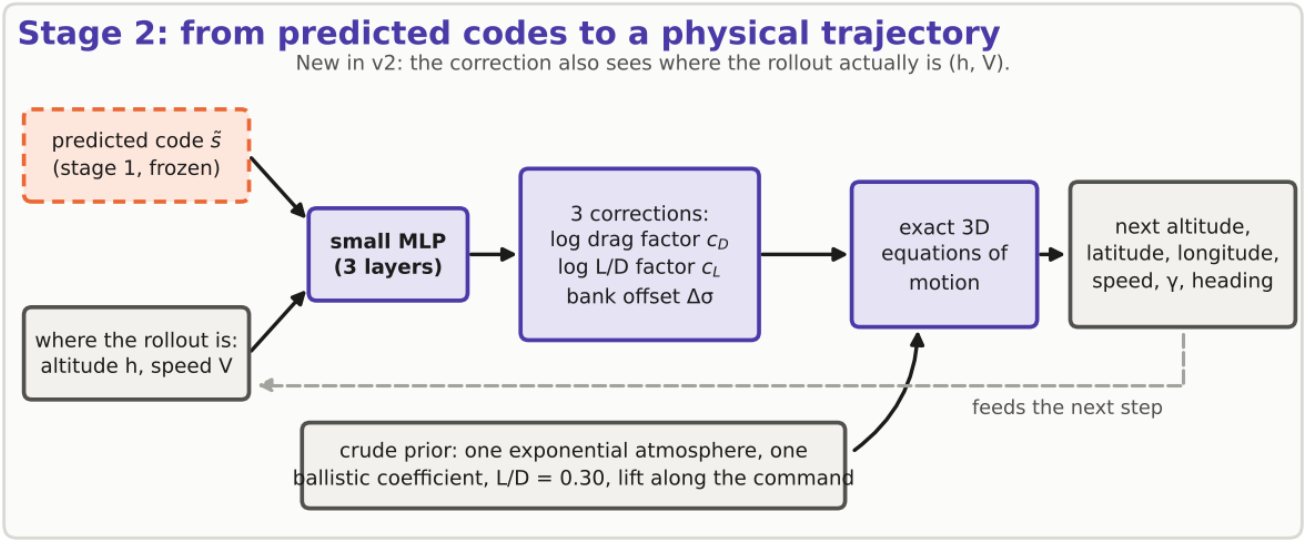


ILLUSTRATION Figure 19. The stage-2 prober. New in v2: besides the predicted code, the small network sees where the rollout actually is (altitude h and speed V). Each step's output feeds the next step.

- **Why add altitude and speed?** The difference between the real atmosphere and the exponential formula depends on altitude, and drag and lift depend on Mach number (speed). Those are properties of where the predicted trajectory is. In dense air the capsule slows down higher up; if the prediction drifts, a correction that ignores the rollout's own altitude is applied in the wrong place. Figure 12 (left) shows how strongly the needed correction varies with altitude.
- **Starts as pure physics:** the network's last layer starts at zero, so before training the prober is the crude prior. That gives the free baseline "physics prior only".
- **Robust loss:** errors are measured in friendly units (1 km of altitude, 5 km of position, 50 m/s, half a degree) through a pseudo-Huber function. It behaves like a squared error for small errors and grows only linearly for large ones, so a few bad long-horizon windows cannot dominate training.
- **Always full precision:** the physics runs in 32-bit floating point even when the networks use faster 16-bit math on the GPU.

17. The ensemble and uncertainty

A single model can be confidently wrong, especially in situations it rarely saw. v2 trains 5 members that differ only in their random starting weights and the random order of their training data. Where the data was plentiful they tend to agree; where it was thin they tend to disagree. Their disagreement is a practical estimate of **epistemic uncertainty**, the uncertainty that comes from limited knowledge.

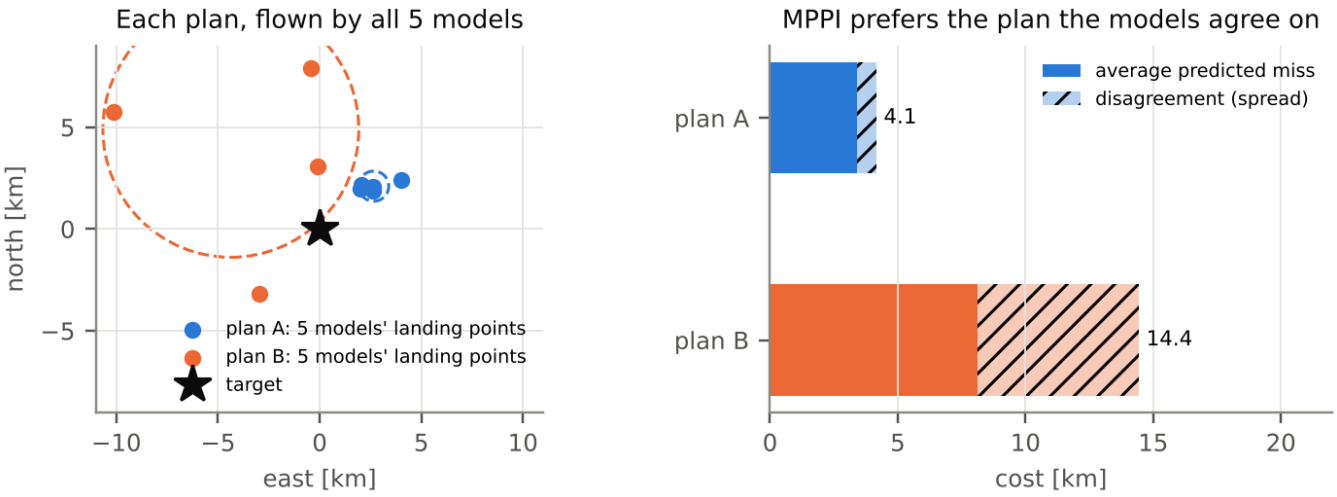
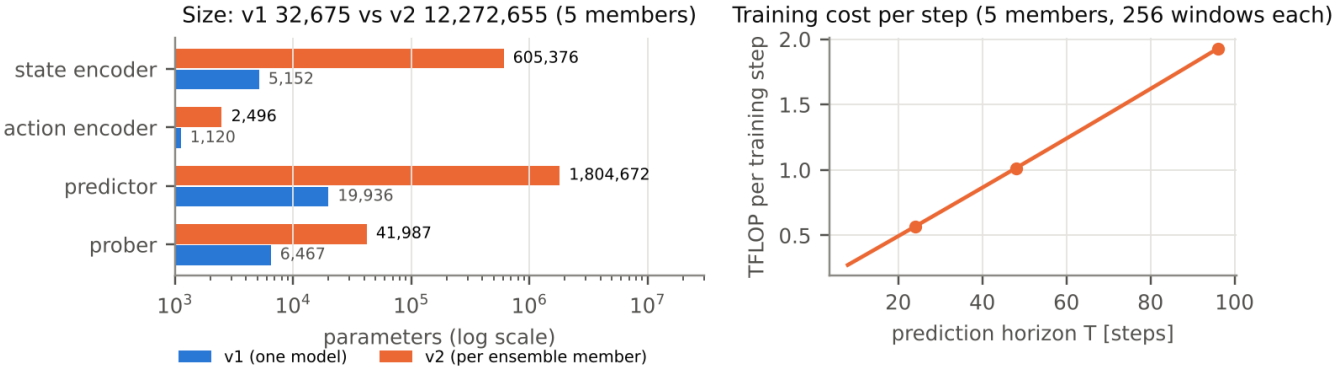


ILLUSTRATION Figure 20. How MPPI uses the ensemble (made-up numbers). Each plan is flown by all 5 models. Plan B's average landing point is fine, but the models disagree widely about it. The cost adds the average miss and the spread, so MPPI prefers plan A, the plan the models agree on.

- Cost of a plan = average over the 5 models of v1's cost, + 1 km per km of spread in the predicted landing points.
- The 5 members are stored as one network whose weights have an extra "member" dimension. Training and planning therefore run all 5 at once, as one set of large matrix multiplications.
- Whether the spread is trustworthy is tested, not assumed: the calibration test (Section 8.4) and the "v2 (no unc.)" control (Section 8.3).

18. Size and compute



FROM THE CODE Figure 21. Left: parameters per component, v1 against one v2 member (log scale). Right: training compute per step as the prediction horizon grows. Longer horizons cost more, which is one reason for the curriculum.

- **2,454,531 parameters per member, 12,272,655 in total**, about 375× v1. Most of them are in the predictor.
- **Mixed precision:** the networks compute in 16-bit (bf16 on L4/A100, fp16 on T4) for speed. Codes, losses, and physics stay in 32-bit.
- **Memory-saving mode** (activation checkpointing) switches on automatically on GPUs under 20 GB, such as the T4. It recomputes some values instead of storing them: about 30% slower, about 10× less memory.
- **Resumable:** every 10 minutes the model, the optimizer, and the training history are saved to Drive.

19. Glossary

Term	Meaning
bank angle σ	how far the capsule is rolled around its direction of flight; the only steering control
drogue parachute	the first, small parachute; it opens at 7.6 km altitude and marks the end of guidance
L/D	lift-to-drag ratio; about 0.3 for Orion
classical predictor-corrector	the hand-built guidance baseline; it re-simulates the flight with a fixed nominal model every second
world model	a model that predicts what happens if a given sequence of actions is applied
policy	a model that maps the current state straight to an action (what RL and cloning learned)
JEPA	Joint-Embedding Predictive Architecture: learns to predict future codes rather than raw data
latent code	the short list of numbers (64 in v2) the encoder uses to summarize the recent flight
encoder / predictor / prober	history → code / code + plan → future codes / codes → physical trajectory
collapse	the failure where every input maps to the same code; prediction becomes trivially perfect and useless
SIGReg	a regularizer that keeps codes spread like a round bell curve, which prevents collapse
λ (lambda)	how strongly SIGReg is weighted against the prediction loss
attention / transformer	a network layer in which every position weighs information from other positions
causal	may only use the past, never the future
curriculum	training on easy (short) problems first, then harder (longer) ones
mask	a 1/0 flag that switches individual loss terms on or off
ensemble	several independently trained models used together; their disagreement signals uncertainty
MPPI	Model Predictive Path Integral control: sample many plans, score them in the model, average the good ones, repeat
macro step	5 seconds; one bank command is held for one macro step
open loop / closed loop	prediction with the true commands fed in / a controller actually flying the simulator
system identification	inferring the unknown conditions (density, L/D) from the observed flight
effective rank	roughly how many independent directions the codes really use (64 = all of them)

Code: `jepa/models_v2.py` (architecture), `jepa/train_v2.py` (training), `jepa/mppi_v2.py` (planner), `jepa/evaluate_v2.py` (diagnostics), `ReEntryAI_JEPA_v2.ipynb` (the experiment). Design notes: `jepa/PLAN.md` §11. This report: `jepa/make_report_v2.py`.